Estructura JavaScript

Lozano R. Angelica M
amlozanor@itc.edu.co

***Resumen*— Este documento describe la estructura funcional, implementación y documentación de una guía interactiva de JavaScript construida durante las clases de Programación III. El trabajo consolida las demostraciones del docente en cinco ejes progresivos: lógica básica, manipulación del DOM, manejo de eventos, uso de la consola y comparativa JavaScript/JQuery. Se empleó HTML5 semántico, CSS3 con variables y JavaScript ES6+, complementado con JQuery 3.7.1 vía CDN. El resultado es un recurso con demos interactivas y un espejo visual de la consola.**

I. INTRODUCCIÓN

En este documento se explica la estructura y el funcionamiento del código JavaScript de la guía de Programación III. El proyecto presenta ejemplos de lógica básica, manipulación del DOM, eventos, mensajes en la consola del navegador y una comparación entre JavaScript puro y jQuery. La finalidad es comprender cómo se conectan los elementos HTML con el código, cómo se reciben los datos del usuario y cómo se muestran los resultados en pantalla o en consola.

Este documento transcribe, organiza y documenta ese proceso, convirtiéndolo en un recurso reproducible y verificable. Los objetivos son: preservar el conocimiento construido en clase, documentar la lógica funcional con estándares formales (JSDoc, formato IEEE), y corregir los errores tipográficos y lógicos detectados durante la transcripción.

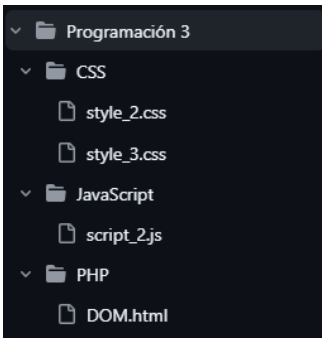
· II. METODOLOGÍA

La transcripción se realizó en tres fases: observación en clase, registrando identificadores HTML, funciones JS y resultados esperados; reconstrucción del código en el editor, respetando la estructura original y añadiendo comentarios JSDoc; y verificación y depuración ejecutando la guía con las herramientas de desarrollador (F12) y contrastando cada salida de consola con la esperada.

III. ARQUITECTURA DEL PROYECTO

A. Estructura de archivos:

La organización sigue el principio HTML = estructura, CSS = presentación, JS = comportamiento:



B. Dependencias externas:

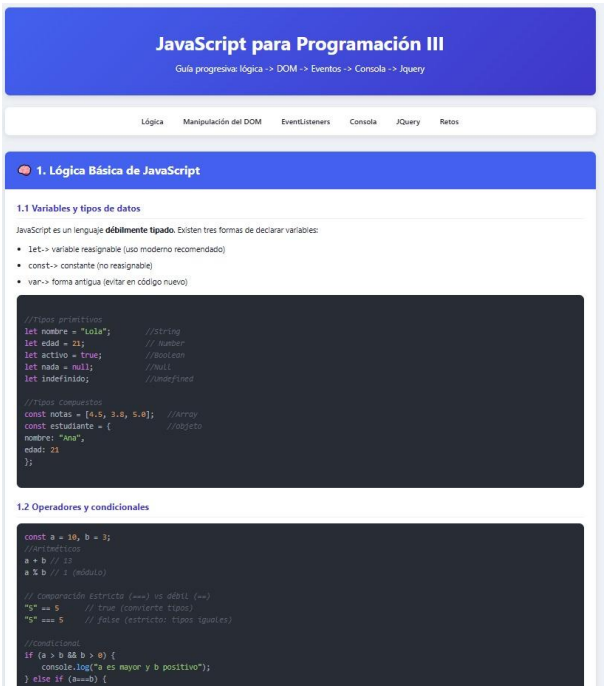
La Tabla I resume las dependencias cargadas vía CDN.

Cuadro I: Dependencias externas del proyecto

Dependencia	Versión	Uso
JQuery	3.7.1	Selección DOM, eventos
Font Awesome	6.4.0	Iconografía

C. Flujo de ejecución:

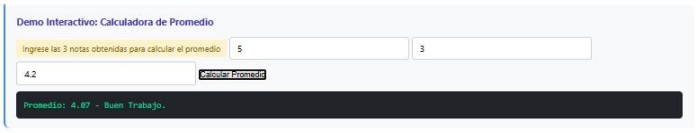
El script script_2.js se carga al final del <body>, garantizando que el DOM esté disponible al ejecutarse los getElementById. Este patrón evita envolver el código en DOMContentLoaded y es adecuado para páginas didácticas pequeñas.



IV. DESCRIPCIÓN FUNCIONAL POR MÓDULOS

A. Calculadora de Promedio

Objetivo: demostrar tipos, operadores y condicionales. Se capturan tres notas con parseFloat, se valida que sean números (isNaN) y estén en [0, 5], se calcula el promedio y se clasifica con if/else if/else (≥ 4.5, ≥ 3.5, ≥ 3.0, resto).



B. Manipulación del DOM

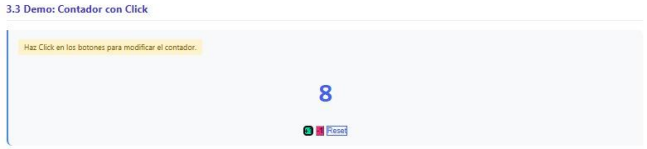
Cuatro funciones globales invocadas desde onclick demuestran las operaciones básicas (Tabla II).

Función	Concepto demostrado
cambiarTexto()	textContent (seguro)
cambiarColor()	element.style.color
cambiarEstilos()	classList.add + style
resetearDemo()	Limpieza de estilos

Cuadro II: Funciones de la sección DOM

C. Event Listeners

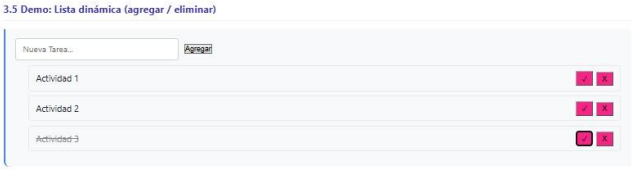
Contador: estado encapsulado en let contador = 0 con tres listeners (btnMas, btnMenos, btnReset).



Entrada en vivo: listener input sobre #inputNombre que refleja valor y longitud en #salidaNombre.



Lista dinámica: btnAgregar crea un <li> con createElement, le añade un con textContent (anti-XSS) y un <button> de eliminación. Se permite agregar con Enter mediante un listener keydown.

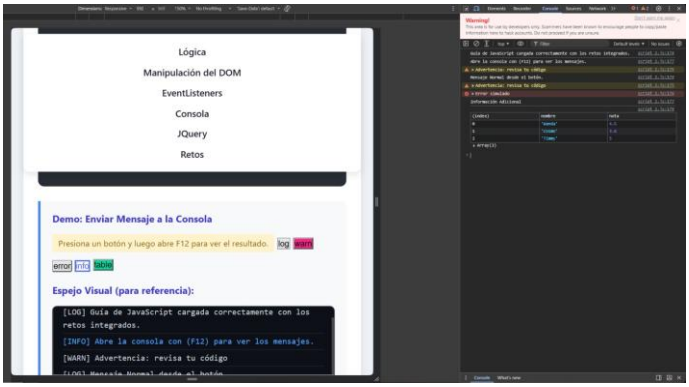


D. Espejo Visual de la Consola

Se guardan referencias originales de console con bind y se sobrescriben para duplicar cada mensaje en #consolaEspejo. La función agregarLineaEspejo formatea los argumentos y aplica una clase según el tipo (log,warn, error, info):

```
const _log = console.log.bind(console);
console.log = (...args) => {
    _log(...args);
    agregarLineaEspejo("log", args.join(" "));
};
```

Listing 1: Intercepción no invasiva de console.log



E. JavaScript vs JQuery

La Tabla III contrasta las sintaxis equivalentes demostradas por los botones #btnJS, #btnJQ y #btnResetJQ.

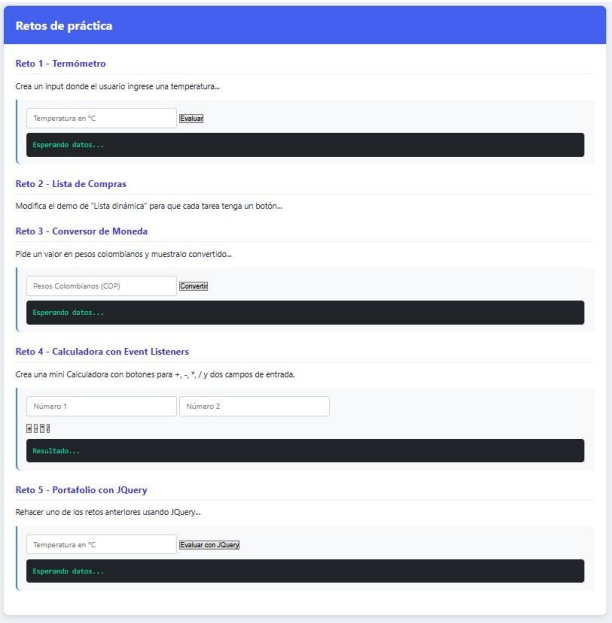
Operación	JS puro	JQuery
Selección	getElementById	$("#x")
Texto	el.textContent	.text()
Estilo	el.style.x	.css()
Evento	addEventListener	.on()

Cuadro III: Equivalencias JavaScript puro / JQuery

V. RETOS DE PRÁCTICA IMPLEMENTADOS

Además de los demos, se implementaron los cinco retos propuestos:

1. Termómetro: clasifica temperatura en frío/templado/caluroso.
2. Lista de compras: botón para marcar tarea como completada (tachado).
3. Conversor de moneda: COP → USD/EUR con tasas ficticias.
4. Calculadora: cuatro operaciones básicas con event listeners.
5. Portafolio con JQuery: termómetro reimplementado con JQuery.



VI. COMPORTAMIENTO ESPERADO EN CONSOLA

La Tabla IV resume las salidas que el estudiante debe verificar durante la práctica.

Acción	Salida esperada
Carga inicial	`[LOG] Guía cargada correctamente`
Promedio válido	`Promedio calculado: 4.50`
Promedio inválido	`[WARN] faltan notas`
Contador	`Contador: N`
Agregar tarea	`Lista: tarea agregada`
Eliminar tarea	`Lista: tarea eliminada`
Botones log/warn/error/info	Mensaje + espejo
Botón table	`Tabla con N fila(s)`

Cuadro IV: Salidas de consola por acción del usuario

VII. BUENAS PRÁCTICAS ADOPTADAS

1. Separación de responsabilidades HTML/CSS/JS.
2. Uso de textContent sobre innerHTML con datos del usuario, mitigando XSS [3].
3. Validación de entradas en calculadora y lista.
4. Encapsulamiento de estado en ámbito de script.
5. Intercepción no invasiva de console preservando comportamiento nativo.
6. Fallback si JQuery no carga (if (window.jQuery)).
7. Documentación JSDoc en funciones públicas.
8. Trazabilidad con console.log/warn/error/info.
9. Accesibilidad básica con aria-label.
10. Diseño responsivo con @media (max-width: 768px).

VIII. CONCLUSIONES

La guía desarrollada en clase y documentada en este trabajo constituye un recurso didáctico completo que cubre la transición de la lógica básica de JavaScript hasta la manipulación del DOM asistida por JQuery. Su valor pedagógico radica en la inmediatez del feedback: cada acción produce una salida visible en pantalla y en consola, reforzando el aprendizaje por observación.

REFERENCIAS

[1] MDN Web Docs, “HTML: A good basis for accessibility,” Mozilla Foundation, 2024. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Learn/Accessibility/HTML

[2] MDN Web Docs, “Manipulating documents,” Mozilla Foundation, 2024. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Manipulating_documents

[3] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet,” 2024. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[4] jQuery Foundation, “jQuery API Documentation,” 2024. [En línea]. Disponible en: https://api.jquery.com/

[5] ECMA International, ECMA-262: ECMAScript Language Specification, 15th ed., Ginebra, Suiza, 2024.

J. Duckett, JavaScript and JQuery: Interactive Front-End Web Development. Indianápolis, IN, EE. UU.: Wiley, 2014.